



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

HTML/CSS e Templating

Angelo Sotgiu

angelo.sotgiu@unica.it

In questa lezione

- Rendering HTML + CSS
- Templating



Recap

- Finora, abbiamo implementato un semplice applicativo web che risponde a delle richieste GET con dei JSON
- Al momento, possiamo interagire con gli endpoint dell'app soltanto eseguendo direttamente richieste HTTP
- Lavoreremo ora sul lato **frontend**, fornendo una semplice interfaccia tramite la quale interagire con l'applicazione
- Le risposte HTTP conterranno dei file HTML, che potranno essere visualizzati da browser
- Vedremo anche come fare in modo che vengano eseguite altre richieste HTTP alla nostra applicazione direttamente tramite le pagine web

Un piccolo test

- Proviamo a scrivere un semplice HTML e darlo in risposta dalla nostra applicazione web. Scriviamo un endpoint sulla root (file main.py) che risponde a richieste HTTP GET:

```
@app.get("/")
def home():
    html = """
    <!DOCTYPE html>
    <html>
        <body>
            <h1>Hello world!</h1>
            <p>Hellooo!</p>
        </body>
    </html>
    """
    return html
```

Un piccolo test

- Cosa succede? <http://127.0.0.1:8000/>
Non funziona come dovrebbe! Perché?
- Analizzando la risposta HTTP, notiamo che il body contiene il nostro HTML, ma l'header Content-Type è application/json
- Questo perché di default FastAPI converte i dati che restituiamo in JSON!
Il browser quindi non è in grado di renderizzare correttamente l'HTML.
- Dobbiamo esplicitamente dire a FastAPI che la nostra risposta è in formato HTML:

```
from fastapi.responses import HTMLResponse
```

```
@app.get("/", response_class=HTMLResponse)
```

File HTML e Templating

- Lavorare sul nostro HTML come stringa in Python è un po' scomodo...
- Spostiamo tutto su un file:
 - creiamo una cartella **templates** dentro app
 - dentro templates, creiamo un file "home.html" e copiamo dentro il nostro HTML
- Per caricare e renderizzare i file HTML, sfrutteremo la funzionalità detta **templating**
- Vedremo nella pratica il significato di questo termine...
- ...per adesso ci basta sapere che è un modo per generare dinamicamente pagine web a partire da file HTML statici
- Esistono diversi tool per templating, useremo Jinja2 (già integrato in FastAPI)
<https://jinja.palletsprojects.com/en/stable/templates/>

Templating

- Modifichiamo in questo modo la nostra funzione endpoint :

```
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates

app = FastAPI()
templates = Jinja2Templates(directory="templates")

@app.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        request=request, name="home.html"
    )
```

Templating

- Complichiamo un po' il nostro HTML, simulando un sito web composto da:
 - titolo (che viene mostrato sulla scheda nel browser)
 - header
 - barra di navigazione
 - contenuto della pagina
 - footer
- L'idea è di riprodurre un pattern comune dove, durante la navigazione:
 - alcuni elementi della pagina restano fissi
 - una porzione di contenuto cambia in base alle azioni eseguite dall'utente
- Creiamo dentro la cartella "template" un file "base.html" con il contenuto della slide seguente


```
<!DOCTYPE html>
<html>
  <head>
    <title>Un sito web</title>
  </head>
  <body>
    <header>
      <h1>Un sito web</h1>
      <p>Un esempio di un sito web.</p>
    </header>
    <nav>
      <a href="/">Home</a>
      <a href="#">Qualcosa</a>
      <a href="#">Qualcos'altro</a>
    </nav>
    <div class="container">
      Qui c'è il contenuto della pagina web.
    </div>
    <footer>
      <p>&copy; 2026 Programmazione Web UNICA </p>
    </footer>
  </body>
</html>
```



Templating - Estensione

- Sfruttiamo adesso la funzionalità di **estensione** dei template per ridurre la duplicazione di codice, evitando di dover riscrivere per ogni pagina web le componenti "fisse"
- Modifichiamo il "base.html" lasciando un placeholder per il contenuto, che potrà essere "riempito" dinamicamente con altri HTML:

```
{% block content %}{% endblock %}
```

- Adesso, estendendo il "base.html" con un nuovo HTML:
 - titolo, header, barra di navigazione e footer saranno ereditati dal template di base, e verranno sempre mostrati senza doverli ridefinire
 - il contenuto sarà quello definito dal nuovo HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Un sito web</title>
  </head>
  <body>
    <header>
      <h1>Un sito web</h1>
      <p>Un esempio di un sito web.</p>
    </header>
    <nav>
      <a href="/">Home</a>
      <a href="#">Qualcosa</a>
      <a href="#">Qualcos'altro</a>
    </nav>
    <div class="container">
      {% block content %}{% endblock %}
    </div>
    <footer>
      <p>&copy; 2026 Programmazione Web UNICA </p>
    </footer>
  </body>
</html>
```

Templating - Estensione

- Modifichiamo adesso il file "home.html" in modo da estendere "base.html". Dovrà contenere all'inizio questa riga:

```
{% extends "base.html" %}
```

- ...e il contenuto aggiuntivo all'interno del blocco desiderato

```
{% block content %}
```

...

```
{% endblock %}
```

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
    <h3 style="text-align: center">Benvenuti in un sito web!</h3>
```

```
{% endblock %}
```

Templating - Variabili

- Possiamo anche "passare" dinamicamente dati dal backend al frontend!
Vediamo subito un piccolo esempio
- Modifichiamo "home.html" così

```
{% extends "base.html" %}  
{% block content %}  
    <h3 style="text-align: center">{{ text }}</h3>  
{% endblock %}
```

- Le doppie parentesi graffe segnalano la presenza di una variabile "text", che dovrà essere fornita dal backend.
Il motore di templating si occuperà di sostituire la variabile con il contenuto ricevuto, e renderizzare la pagina web.

Templating - Variabili

- Dal backend, dovremo fornire un dizionario "context" che contiene il nome delle variabili attese dal template come chiavi e il loro contenuto come valori.

Nel nostro esempio:

```
@app.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        request=request, name="home.html",
        context={"text": "Benvenuti in un sito web!"}
    )
```

- Verifichiamo il corretto funzionamento <http://127.0.0.1:8000/>



Templating - Escaping

- ATTENZIONE! Dato che le variabili fornite nel contesto sono automaticamente trasformate in stringhe e messe dentro l'HTML, se il contenuto delle variabili è esso stesso HTML verrà interpretato come tale!
- Questo può avere effetti indesiderati, e soprattutto creare vulnerabilità se un'attaccante è in grado di alterare di proposito questi dati!
Ad esempio, inserendo degli script malevoli, che verrebbero poi eseguiti.
- Per evitare questo, dobbiamo sanitizzare i dati con l'operazione di escaping (ovvero, i caratteri speciali come <, >, &, etc. saranno codificati come normali stringhe e non come codice HTML.
- Di default, Jinja2 ha l'escaping automatico disabilitato per ragioni computazionali, ma le librerie che lo usano internamente (come FastAPI) lo abilitano. Se voglio disabilitare questa opzione per qualche motivo (a mio rischio e pericolo...), posso usare il seguente filtro:

```
<p>{{ text|safe }}</p>
```

Templating - Variabili con attributi

- Possiamo fornire qualsiasi tipo di dato nel contesto, verrà convertito in stringa.
- Inoltre, è possibile accedere ai suoi attributi direttamente nel template. Esempio (file "home.html", al posto dell'h3):

```
<div style="text-align: center">  
  <h1>{{ text.title }}</h1>  
  <p>{{ text.content }}</p>  
</div>
```

```
@app.get("/", response_class=HTMLResponse)  
def home(request: Request):  
    text = {  
        "title": "Benvenuti in un sito web!",  
        "content": "Forse è meglio che lo faccio riscrivere a Claude..."  
    }  
    return templates.TemplateResponse(  
        request=request, name="home.html", context={"text": text}  
    )
```


Templating – Operazioni avanzate

- Tramite la seguente sintassi possiamo sfruttare diverse funzionalità avanzate di Jinja2
`{% ... %}` -> al posto dei puntini inseriremo il costrutto necessario
- Esempio: ciclo **for**

```
<ul>  
    {% for word in sequence %}  
        <li>{{ word }}</li>  
    {% endfor %}  
</ul>
```

- Nella funzione endpoint dovremmo aggiungere nel context:

```
context={"text": text, "sequence": ["a", "b", "c"]}
```

Templating – Operazioni avanzate

- La sintassi dentro il for è simile a quella di Python.
Ad esempio, sui dizionari possiamo iterare su chiavi e valori, oppure solo sulle chiavi:

```
{% for key, value in dictionary.items() %}  
  <p>Key: {{ key }} Value: {{ value }}</p>  
{% endfor %}
```

oppure

```
{% for key in dictionary %}  
  <p>Key: {{ key }} Value: {{ text[key] }}</p>  
{% endfor %}
```

- Nella funzione endpoint dovremo aggiungere nel context:

```
context={"text": text, "dictionary": {"a": 0, "b": 1, "c": 2}}
```



Struttura sito fittizio

- Simuliamo adesso un sito web di un negozio composto da:
 - una home page
 - una pagina con l'elenco dei prodotti in vendita
- Utilizzeremo il "base.html" (cambiando il testo e i link al suo interno) e lo estenderemo per avere le due pagine.
- Modifichiamo la funzione di endpoint per la homepage così:

```
@app.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        request=request, name="home.html",
        context={"text": "Welcome to the store"}
    )
```

FastAPI – File statici

- Modifichiamo anche l'homepage aggiungendo un'immagine.
I file statici (CSS, immagini, etc.) dovrebbero essere contenuti in un'apposita cartella:
 - creiamo una cartella "static"
 - copiamoci dentro un'immagine qualsiasi
- Dobbiamo adesso configurare FastAPI in modo che monti la cartella static.
Aggiungiamo nel main.py il seguente import:

```
from fastapi.staticfiles import StaticFiles
```

e questo comando dopo la creazione dell'oggetto app:

```
app.mount("/static", StaticFiles(directory="static"), name="static")
```

FastAPI – File statici

- Possiamo adesso modificare il file "home.html" in questo modo:

```
{% extends "base.html" %}
{% block content %}
    <h3 style="text-align: center">{{ text }}</h3>
    <br>
    <div style="text-align: center">
        
    </div>
{% endblock %}
```

- Dentro le doppie parentesi graffe possiamo anche chiamare delle funzioni.
In questo caso stiamo chiamando il metodo "url_for", che recupera il percorso di una risorsa esposta dalla nostra applicazione web, ovvero la cartella "/static", e il file indicato nel parametro "path".

File CSS

- L'interfaccia fa un po' schifo... proviamo ad aggiungere un file CSS
 - creiamo un file "styles.css" dentro la cartella "static"
 - riempiamolo con qualche semplice configurazione (continua nella prossima slide)

```
* {  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
}  
  
body {  
    font-family: Arial, sans-serif;  
    background-color: #f2f2f2;  
    color: #333;  
    line-height: 1.6;  
}
```

- Qualche altra configurazione...

```
nav {  
  background: #005599;  
  display: flex;  
  justify-content: center;  
  padding: 10px;  
}  
nav a {  
  color: #fff;  
  text-decoration: none;  
  margin: 0 15px;  
  font-size: 16px;  
  transition: color 0.3s;  
}
```

```
nav a:hover {  
  color: #ffcc00;  
}
```

```
.container {  
  max-width: 1200px;  
  margin: 20px auto;  
  padding: 20px;  
  background-color: #fff;  
}
```

```
header {  
  background: #003366;  
  color: #fff;  
  padding: 20px;  
  text-align: center;  
}
```

```
footer {  
  background: #003366;  
  color: #fff;  
  text-align: center;  
  padding: 15px;  
  margin-top: 20px;  
}
```

- Infine, aggiungiamo il link al file CSS nel file "base.html":

```
<head>  
  ...  
  <link rel="stylesheet" href="{{ url_for('static', path='/styles.css') }}">  
</head>
```



Pagina con elenco prodotti

- Aggiungiamo adesso una nuova funzione endpoint all'applicazione per la nuova pagina web, definendo anche una lista di prodotti fittizi con campi:
 - name
 - price
 - location

```
products = [  
    {"name": ..., "price": ..., "location": ...},  
]  
  
@app.get("/products", response_class=HTMLResponse)  
def products(request: Request):  
    return templates.TemplateResponse(  
        request=request, name="products.html",  
        context={"product_list": product_list}  
    )
```


Pagina con elenco prodotti

- Creiamo adesso un template "products.html" per mostrare la lista di prodotti in una tabella, costruita dinamicamente:

```
{% extends "base.html" %}
{% block content %}
<table>
  <tr>
    <th>Name</th>
    <th>Price</th>
    <th>Location</th>
  </tr>
  {% for p in product_list %}
    <tr>
      <td>{{ p.name }}</td>
      <td>{{ p.price }}</td>
      <td>{{ p.location }}</td>
    </tr>
  {% endfor %}
</table>
{% endblock %}
```

Pagina con elenco prodotti

- Nel CSS aggiungiamo...

```
table {  
    width: 50%;  
    margin: 0 auto;  
    border-collapse: collapse;  
}
```

```
th, td {  
    padding: 10px;  
    border: 1px solid #ddd;  
}
```

```
th {  
    background-color: #f5f5f5;  
}
```

- Aggiungiamo anche un link alla nuova pagina nella barra di navigazione, nel file "base.html":

```
<a href="{{ url_for('products') }}">Products</a>
```